



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Verso la Validazione Automatica: un'Analisi Empirica di Tool per la Generazione di Scenari di Guida

RELATORE

Prof. Carmine Gravino

Prof. Dario Di Nucci

Dott. Antonio Trovato

Università degli Studi di Salerno

CANDIDATO

Raffaele Coppola

Matricola: 0512116487

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

It is our choices that show what we truly are, far more than our abilities

Abstract

La sicurezza stradale rappresenta tuttora una sfida globale: nonostante i progressi tecnologici e normativi, il numero di incidenti mortali rimane elevato e i loro impatti sociali ed economici continuano a pesare in maniera significativa. Parallelamente, il mercato automobilistico sta vivendo una transizione che vede i sistemi di guida automatizzata sempre più presenti nei veicoli di nuova generazione. Tale evoluzione porta con sé grandi promesse in termini di riduzione del rischio, ma richiede altrettanta urgenza nell'affrontare il problema della validazione: dimostrare che gli Automated Driving Systems (ADS) siano realmente sicuri.

La simulazione rappresenta oggi il canale privilegiato per affrontare questo obiettivo, consentendo di riprodurre in sicurezza e su larga scala scenari rari o potenzialmente pericolosi. In questa tesi viene proposta un'analisi empirica di tre strumenti di generazione automatica di scenari, *ScenarioFuzz-LLM*, *TM-Fuzzer* e *SimADFuzz*, valutandone il comportamento lungo tre dimensioni chiave: *criticità*, *diversità* ed *efficacia*. L'approccio combina un framework di logging dedicato e indicatori quantitativi per confrontare sistematicamente le capacità dei diversi generatori.

I risultati mostrano profili complementari: strumenti focalizzati su scenari ad alta intensità di rischio, altri orientati a varietà e bilanciamento, altri ancora a test di robustezza prolungata. Da ciò emerge che non esiste un singolo approccio dominante, ma strategie differenziate da adottare in funzione degli obiettivi di validazione. La discussione si conclude con i limiti metodologici emersi ed apre a possibili sviluppi futuri, come l'introduzione di metriche di realismo e l'integrazione di ADS completi.

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	iv
1 Introduzione	1
1.1 Contesto applicativo	1
1.2 Motivazioni ed obiettivi	2
1.3 Risultati	3
1.4 Struttura della tesi	3
2 Background e Stato dell'Arte	4
2.1 Dalla guida assistita alla guida autonoma	5
2.2 Real world testing	7
2.3 Simulation based testing	8
2.3.1 Approcci di generazione	9
2.3.2 I simulatori	11
2.4 Related works	13
3 Metodologia	14
3.1 Research Questions	15
3.2 Tool analizzati	15

3.2.1	SimADFuzz	17
3.2.2	TM-Fuzzer	18
3.2.3	ScenarioFuzz-LLM	18
3.3	Raccolta dei dati	20
3.3.1	System under stress: BehaviorAgent	20
3.3.2	Sistema di logging	21
3.3.3	Arricchimento dei log	22
3.3.4	Pipeline di clustering	23
3.3.5	Integrazione del framework nei tool selezionati	24
3.4	Metodologia di valutazione	25
3.4.1	Metriche e punteggi	25
4	Analisi dei risultati	30
4.1	Valutazione della criticità	31
4.1.1	Profilo di criticità e punteggio aggregato	31
4.1.2	Analisi del rischio latente	33
4.2	Valutazione della diversità	35
4.2.1	Analisi dei cluster	35
4.3	Valutazione dell'efficacia	37
5	Conclusioni e Lavori Futuri	39
	Bibliografia	42

Elenco delle figure

2.1	Diagramma architetturale di Autoware [1].	7
4.1	Profilo di criticità comparativo dei tool	32
4.2	Metriche di rischio latente	33
4.3	Selezione del numero ottimale di cluster	36
4.4	Cluster e distribuzione degli scenari	36
4.5	Punteggio di efficacia comparativo dei tool	37
4.6	Distribuzione complessiva degli outcome	38

Elenco delle tabelle

3.1	Metriche, valori massimali M_{max} e pesi adottati nel calcolo di $C(T)$. .	28
3.2	Sintesi delle metriche considerate e significato dei valori.	29
4.1	Risultati metriche di criticità	32
4.2	Statistiche rischio latente	34
4.3	Risultati metriche diversità	35

CAPITOLO 1

Introduzione

1.1 Contesto applicativo

Gli incidenti stradali rappresentano una delle principali cause di morte a livello globale. Secondo l'*Organizzazione Mondiale della Sanità*, nel 2023 circa 1.19 milioni di persone hanno perso la vita a causa di sinistri stradali, costituendo la prima causa di morte tra bambini e giovani adulti di età compresa tra i 5 e i 29 anni ¹. A tali perdite si aggiungono ingenti costi economici e sociali, stimati in circa il 3% del PIL per molti paesi.

Per fronteggiare questa sfida, negli ultimi anni si è assistito ad una crescente diffusione di sistemi avanzati di assistenza alla guida (ADAS) e dei sistemi di guida autonoma (ADS), con l'obiettivo di ridurre drasticamente la probabilità e la gravità degli incidenti.

Nonostante tali progressi, i sistemi di guida autonoma non sono esenti da criticità. La *National Highway Traffic Safety Administration (NHTSA)*, attraverso il programma di monitoraggio *Standing General Order on Crash Reporting*, ha documentato tra luglio 2021 ed agosto 2025 oltre 1500 incidenti che hanno coinvolto veicoli con un sistema

¹World Health Organization: Road traffic injuries, accessed on 22/09/2025

ADS attivo nei 30 secondi antecedenti l’impatto ². Tali dati, tuttavia, vanno interpretati con cautela: la semplice classificazione di un incidente come *ADS engaged* non implica che il sistema di guida autonoma sia la causa primaria dell’evento. Inoltre, le statistiche diffuse dalla NHTSA si riferiscono esclusivamente al contesto statunitense e non possono essere considerate rappresentative dell’adozione e performance di tali sistemi a livello globale.

1.2 Motivazioni ed obiettivi

La validazione dei sistemi di guida autonoma richiede la disponibilità di scenari di test vari e significativi. Tuttavia, il solo ricorso alla sperimentazione su strada comporta costi elevati, tempi poco sostenibili e non consente di coprire adeguatamente i cosiddetti *corner cases*, ovvero situazioni rare ma potenzialmente pericolose che rappresentano il vero banco di prova per un ADS.

In questo contesto, la simulazione offre un ambiente sicuro e scalabile per riprodurre milioni di chilometri di guida, variando in modo controllato condizioni stradali, meteorologiche e comportamento degli altri utenti della strada. Diventa così cruciale il ruolo degli strumenti di generazione di scenari, che consentono di automatizzare la creazione di scenari complessi ed aumentare la probabilità di individuare vulnerabilità.

L’obiettivo di questo lavoro è condurre un’analisi empirica dei principali strumenti di generazione automatica di scenari, osservandone il comportamento attraverso l’evidenza sperimentale. L’attenzione si concentra su tre dimensioni fondamentali. La prima è la *criticità*, intesa come capacità di produrre scenari realmente sfidanti, che inducano collisioni, violazioni o situazioni di rischio per l’ADS. La seconda è la *diversità*, ovvero la varietà degli scenari generati: un tool efficace non dovrebbe limitarsi a ripetere configurazioni simili. Infine, viene considerata l’*efficacia*, definita come il rapporto tra il numero di scenari effettivamente utili alla validazione e il totale di quelli generati.

²NHTSA - Standing General Order on Crash Reporting, accessed on 22/09/2025

1.3 Risultati

Sono stati confrontati tre strumenti diversi di generazione di scenari, dai quali sono emersi altrettanti profili distinti: *ScenarioFuzz-LLM* si è dimostrato particolarmente efficace nel generare collisioni e violazioni, offrendo quindi scenari fortemente critici e adatti a testare la resilienza immediata dell'ADS. *TM-Fuzzer* ha mostrato un equilibrio tra diversità ed efficacia, riuscendo a coprire in maniera bilanciata tipologie di scenari differenti senza rinunciare a una buona frequenza di fallimenti. *SimADFuzz*, infine, ha prodotto scenari lunghi e consistenti, caratterizzati da minore incidenza di collisioni ma utili a osservare la tenuta funzionale dell'agente nel tempo. Questi risultati suggeriscono che non esista un singolo strumento superiore, ma piuttosto approcci complementari che rispondono a obiettivi diversi della validazione.

1.4 Struttura della tesi

La tesi è organizzata in cinque capitoli principali.

- **Capitolo 1 - Introduzione:** definisce il contesto applicativo, le motivazioni e gli obiettivi, i risultati principali e la struttura complessiva del lavoro.
- **Capitolo 2 - Background e Stato dell'arte:** introduce i concetti di ADAS e ADS, descrive il ruolo della simulazione, presenta la tassonomia dei metodi di generazione di scenari e i simulatori più diffusi, concludendo con i lavori correlati.
- **Capitolo 3 - Metodologia:** illustra le domande di ricerca, i criteri di selezione dei tool, l'architettura di logging e analisi, la pipeline di clustering e le metriche adottate per la valutazione.
- **Capitolo 4 - Analisi dei risultati:** discute i dati sperimentali confrontando i tre strumenti selezionati nelle dimensioni di criticità, diversità ed efficacia e fornisce le risposte alle research questions.
- **Capitolo 5 - Conclusioni e lavori futuri:** sintetizza i risultati emersi, evidenzia i limiti della sperimentazione e propone possibili sviluppi, tra cui l'integrazione di ADS completi e l'introduzione di metriche di realismo.

CAPITOLO 2

Background e Stato dell'Arte

In questo capitolo vengono introdotti i concetti fondamentali che costituiscono il quadro teorico di riferimento. Si apre con una panoramica sui livelli di automazione definiti dallo standard SAE J3016, chiarendo la distinzione tra sistemi di assistenza alla guida (ADAS) e sistemi di guida autonoma (ADS), insieme a una sintesi delle loro architetture principali. Segue l'analisi delle strategie di validazione, a partire dal real world testing e dai suoi limiti in termini di costi, sicurezza e copertura scenariale, fino all'affermazione del simulation-based testing come alternativa più efficace ed efficiente. All'interno di questo contesto vengono approfonditi i concetti di scenario e le principali tassonomie di approcci per la generazione, distinguendo tra metodi data-driven, adversarial e knowledge-based. La sezione successiva è dedicata ai simulatori, con un focus su CARLA e sul ruolo della co-simulazione. Il capitolo si chiude con l'analisi dei contributi più rilevanti in letteratura, come il workshop SBFT e la piattaforma SafeBench, che costituiscono riferimenti concettuali per la metodologia adottata in questo lavoro.

2.1 Dalla guida assistita alla guida autonoma

Data la complessità e la varietà dei sistemi di guida autonoma, è emersa la necessità di una tassonomia condivisa che permetta di distinguere in modo chiaro i diversi livelli di automazione [2]. A questo scopo la *Society of Automotive Engineers* ha introdotto lo standard SAE J3016¹, oggi riconosciuto a livello mondiale, che definisce sei livelli progressivi di automazione. Nei paragrafi seguenti vengono presentati i principali elementi di tale classificazione, insieme a una breve introduzione ai concetti di ADAS e, in particolare, di ADS.

Livelli di automazione Lo standard individua sei livelli distinti, che vanno dall'assenza totale di supporto fino alla completa autonomia del veicolo:

0. Nessuna automazione della guida

Tutte le funzioni di guida sono a carico del conducente.

1. Assistenza alla guida

Il sistema supporta il conducente controllando in modo continuo esclusivamente la sterzata o esclusivamente l'accelerazione/frenata.

2. Automazione parziale della guida

Il sistema controlla contemporaneamente sterzata e accelerazione/frenata, ma il conducente deve monitorare costantemente e intervenire se necessario.

3. Automazione condizionata della guida

Il sistema gestisce la guida in scenari specifici (ad esempio autostrada, traffico), richiedendo però la disponibilità immediata del conducente a subentrare.

4. Alta automazione della guida

Il sistema gestisce interamente la guida e può fronteggiare guasti portando il veicolo in condizioni di sicurezza.

5. Automazione completa della guida

Il sistema guida in modo autonomo in qualsiasi contesto, senza limiti operativi né necessità di intervento umano.

¹https://www.sae.org/standards/content/j3016_202104/, accessed on 23/09/2025

A partire da questa suddivisione in livelli è possibile classificare, e quindi definire, in maniera più esplicita e chiara, la distinzione tra i sistemi di assistenza alla guida e i sistemi di guida autonoma.

ADAS Con il termine Advanced Driver Assistance Systems (ADAS) si indica un insieme eterogeneo di tecnologie a supporto del conducente. Tali sistemi possono includere sia funzionalità di automazione della guida fino al livello 2 dell'attuale tassonomia, sia sistemi di sicurezza attiva che, pur essendo automatizzati, non rientrano nella classificazione della guida automatizzata.

Questi ultimi, infatti, sono progettati per intervenire solo in situazioni puntuali di pericolo. Esempi tipici sono il Lane Keeping Assistance (LKA) o l'Automatic Emergency Braking (AEB), che forniscono correzioni momentanee senza modificare il ruolo centrale del conducente nella guida. [2, 3]

ADS Con il termine Automated Driving System (ADS), d'altro canto, si fa riferimento a un sistema di guida automatizzata conforme ai livelli 3, 4 e 5. A questi livelli il sistema non si limita a supportare il conducente, ma assume progressivamente un ruolo centrale nella conduzione del veicolo, fino a raggiungere, al livello 5, una completa automazione senza supervisione umana [2, 3].

Cenni sull'architettura Un ADS è tipicamente articolato in quattro moduli funzionali principali: *sensing*, *perception*, *planning* e *control*. Il *sensing module* raccoglie i dati sull'ambiente circostante tramite sensori dedicati (LiDAR, radar, telecamere, ecc.). Queste informazioni vengono poi elaborate dal *perception module*, che ne estrae le caratteristiche rilevanti e le rende disponibili al *planning module*. Quest'ultimo ha il compito di generare la traiettoria di guida ottimale sulla base del contesto percepito, mentre il *control module* interviene sul veicolo per far sì che segua la traiettoria pianificata [2].

Nei sistemi più recenti si sta diffondendo anche un approccio alternativo, detto *end-to-end*, in cui i moduli di *perception*, *planning* e *control* vengono accorpati in un unico macro-modulo, spesso implementato mediante modelli di deep learning.

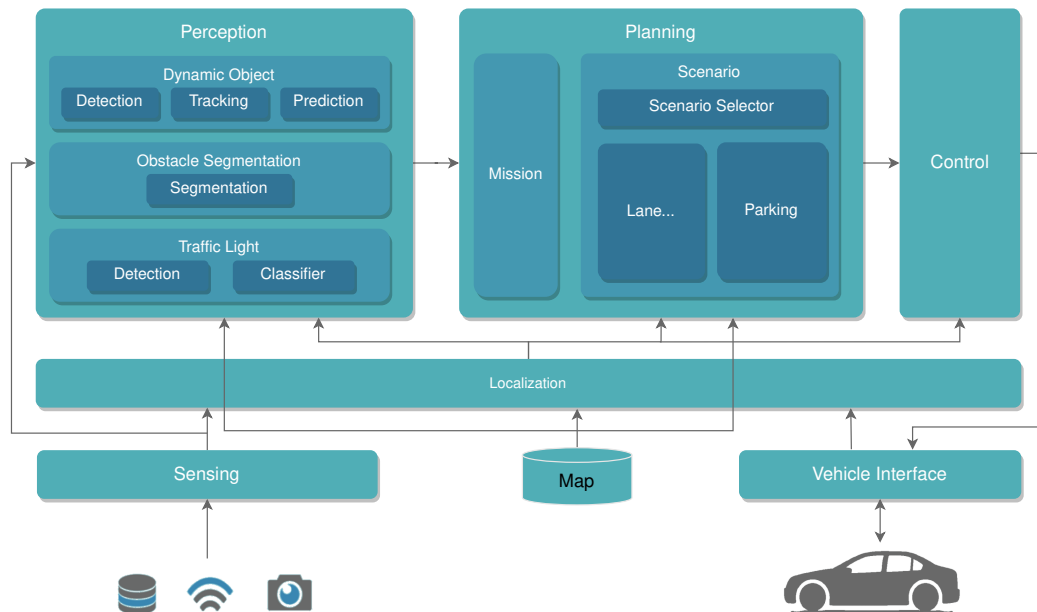


Figura 2.1: Diagramma architetturale di Autoware [1].

2.2 Real world testing

Una volta chiariti i livelli di automazione e le architetture degli ADS, è necessario analizzare le strategie di validazione. Storicamente, la prima forma di validazione è stata il *real world testing*, ovvero la sperimentazione diretta su strada.

Nei test su strada i veicoli autonomi vengono collaudati su strade pubbliche, dove possono verificarsi situazioni e condizioni impreviste. Per dimostrare l'affidabilità degli ADS, sono necessarie percorrenze molto elevate e l'indicatore comunemente adottato a tale scopo è il numero di chilometri percorsi senza intervento umano, ovvero i *kilometres per disengagement*. Nel 2020 i veicoli di Waymo², uno dei principali attori nel panorama mondiale di sviluppo di guida autonoma, hanno totalizzato oltre un milione di chilometri su strade pubbliche, con una media di circa 48000 km per ciascun disengagement [4, 5]. Un livello di sperimentazione di questa portata non è, però, realistico per la maggior parte delle aziende: solo l'8% dei praticanti ha superato i 100000 km di test su strada, mentre l'11% si è fermato tra i 10000 e i 50000 km [5]. Questi esperimenti, spesso chiamati *Naturalistic Field Operational Tests (N-FOT)* [6, 7], permettono di osservare gli ADS in condizioni naturali, ma richiedono percorrenze enormi per registrare eventi rari. Negli Stati Uniti, ad esempio, si stima

²<https://waymo.com/>, accessed on 23/09/2025

che siano necessarie circa 0,53 milioni di miglia per osservare un incidente con referto di polizia e oltre 99 milioni di miglia per un incidente fatale, motivo per cui diversi studi riportano che un N-FOT non può essere condotto con un budget inferiore a 10 milioni di dollari, senza considerare i rischi per i tester e gli utenti della strada [6]. Oltre al volume di percorrenza e ai costi, rimane aperta la questione della copertura scenariale. Le condizioni di traffico variano in modo significativo e non sono facilmente riproducibili in un singolo programma di test. Questo limita, inevitabilmente, la possibilità di sottoporre il sistema agli stessi stimoli in modo controllato e quindi di valutare in maniera sistematica la relazione tra input e output del veicolo autonomo. Dati i seguenti motivi, i test su strada, pur essendo considerati la pratica idealmente migliore per la validazione di questi sistemi così complessi, risultano costosi, pericolosi e poco efficaci nel dimostrare che i veicoli autonomi siano realmente più sicuri dei conducenti umani. Risulta quindi essenziale un metodo alternativo di testing che consenta una validazione e verifica efficace ma soprattutto efficiente.

2.3 Simulation based testing

Le limitazioni intrinseche del testing su strada hanno portato alla diffusione del simulation-based testing, oggi considerato uno strumento fondamentale per valutare gli ADS in modo sicuro, efficiente e sistematico. Come riportato da G. Lou *et al.* [5], le principali ragioni che spingono oggi i produttori di ADS ad investire nella ricerca basata sul *simulation-based testing* possono essere riassunte nei seguenti punti:

- Gli ambienti di simulazione moderni consentono di testare corner cases che, nel mondo reale, risulterebbero troppo onerosi da riprodurre.
- Per scenari di guida pericolosi, come le collisioni, la simulazione garantisce un livello di sicurezza irraggiungibile nei test reali.
- Nel processo di sviluppo di un ADS, il *regression testing* è cruciale: la simulazione consente agli sviluppatori di evitare il *deploy* di ogni singolo *commit* sul veicolo reale.

Risulta utile, quindi, chiarire alcuni concetti chiave che costituiscono la base della rappresentazione virtuale di situazioni di guida.

Scenario In letteratura il termine scenario è utilizzato in diversi ambiti disciplinari, ma nel contesto della guida automatizzata esso indica l'evoluzione temporale di più scene collegate tra loro da azioni ed eventi, includendo obiettivi e vincoli che ne guidano lo sviluppo. Nella metafora teatrale comunemente adottata, lo scenario corrisponde a una sorta di trama composta da scene e sequenze di azioni, con un grado di dettaglio che può variare da descrizioni sintetiche fino a veri e propri storyboard [8]. La ricerca sul tema propone tipicamente una classificazione del concetto di scenario in tre categorie principali: *funzionali*, definiti in linguaggio naturale e utili nelle prime fasi concettuali; *logici*, rappresentati tramite variabili di stato e intervalli di valori; e *concreti*, che assegnano valori specifici ai parametri e possono essere direttamente utilizzati come casi di test [9].

2.3.1 Approcci di generazione

La letteratura sul tema propone una grande varietà di metodi per la generazione di scenari, spesso descritti con classificazioni eterogenee e non sempre confrontabili tra loro. Per garantire coerenza e chiarezza nell'analisi, in questo lavoro si è deciso di adottare come riferimento principale la tassonomia proposta da *Ding et al.* [10], che distingue gli approcci in tre macro-categorie: *data-driven*, *adversarial* e *knowledge-based*.

Data-Driven Generation L'approccio *data-driven* si fonda sull'impiego di dati reali per la costruzione di scenari di test. Tali scenari possono essere semplicemente riprodotti a partire dai log disponibili o arricchiti mediante tecniche di clustering e perturbazioni casuali; in alternativa, si possono utilizzare modelli probabilistici e generativi capaci di apprendere la distribuzione dei dati e di sintetizzare nuove istanze non osservate nei dataset originari. Questo consente di ottenere scenari caratterizzati da un elevato grado di realismo, ma limita la capacità di rappresentare eventi rari e situazioni effettivamente critiche, che risultano scarsamente documentati nei dati raccolti.

Adversarial Generation L'approccio *adversarial* consiste nel generare scenari rischiosi che hanno lo scopo di mettere in difficoltà l'ADS, includendolo direttamente nel ciclo di generazione (*vehicle in the loop*). In letteratura si distinguono due tipologie principali:

1. **Static:** gli scenari statici prendono di mira i moduli di percezione dell'ADS, operando su input a singolo frame come immagini o dati LiDAR.
2. **Dynamic:** gli scenari dinamici, invece, si sviluppano nel tempo e sono utilizzati per testare i moduli di pianificazione e controllo. La loro generazione può avvenire controllando le condizioni iniziali della simulazione (ad esempio velocità e posizione degli agenti circostanti) oppure mediante lo sviluppo di politiche avversarie basate su tecniche di Reinforcement Learning, in cui l'agente avversario adatta le proprie azioni passo dopo passo per massimizzare la probabilità di situazioni critiche.

Knowledge-based generation L'approccio *knowledge-based* si distingue dai due precedentemente citati in quanto integra nel processo di generazione la conoscenza esterna intesa come regole del traffico, leggi fisiche o vincoli derivanti dall'esperienza umana. Anche per questo tipo di approccio vengono definiti due sotto-approcci:

1. Il primo adotta regole predefinite, in cui scenari e condizioni vengono progettati manualmente da esperti per innescare eventi specifici, ad esempio collisioni o manovre rischiose. Questo approccio si basa su descrizioni formali, ontologie o descrizioni testuali in linguaggio naturale.
2. Il secondo si colloca, invece, nell'ambito del *knowledge-guided learning* in cui regole esplicite vengono usate per guidare modelli di machine learning nella generazione. Alcuni studi introducono regole semantiche, come il divieto di sovrapposizione tra veicoli o l'obbligo di rispettare la direzione delle corsie. Altri rappresentano le relazioni tra gli oggetti dello scenario attraverso grafi causali, ovvero strutture che modellano i legami di causa-effetto tra eventi ed attori. Un'ulteriore declinazione è l'ottimizzazione vincolata, che applica *metodi evolutivi* o euristici per controllare i comportamenti degli agenti circostanti e

ridurre lo spazio percorribile dal veicolo ego, aumentando così la probabilità di situazioni critiche.

Queste strategie di generazione non sono applicate in astratto, ma si collocano all'interno di specifici livelli di validazione dell'ADS. In letteratura si distinguono due categorie principali: il *module testing*, focalizzato sull'analisi mirata di singoli moduli sottoposti a condizioni di stress controllate, e il *system testing*, che adotta una prospettiva più olistica per individuare situazioni in grado di compromettere l'intero sistema [2]. In questo quadro, gli *adversarial attack*, spesso usati per testare la robustezza del modulo di percezione, sono riconducibili all'approccio *adversarial*. Per quanto riguarda invece il *system testing*, l'attenzione si è concentrata prevalentemente sulla strategia di generazione nota come *search-based generation*.

Search-based generation La generazione basata sulla ricerca potrebbe essere vista come un approccio generativo differente da quelli precedentemente citati [2], ma anche come l'effettiva concretizzazione degli approcci *adversarial* e *knowledge-based*. In questo contesto, il problema viene formalizzato come un processo di ottimizzazione nello spazio dei parametri, con l'obiettivo di individuare configurazioni che massimizzino (o minimizzino) una funzione obiettivo. Tra le tecniche più utilizzate rientrano gli algoritmi genetici, che si ispirano ai principi evolutivi per esplorare lo spazio di ricerca in maniera efficiente. Un algoritmo particolarmente affermato in questo ambito è il NSGA-II (Non-dominated Sorting Genetic Algorithm II), molto impiegato per affrontare problemi di ottimizzazione multi-obiettivo [11].

2.3.2 I simulatori

Nel contesto del *simulation-based testing* per gli ADS, i simulatori più diffusi e utilizzati sono senza dubbio CARLA [12] e LGSVL [13].

Co-simulazione Un aspetto rilevante è la possibilità di utilizzare la *co-simulazione*, ovvero l'accoppiamento di più simulatori specializzati in domini diversi, così da ottenere una valutazione olistica del sistema di riferimento. Un esempio rappresentativo è l'integrazione tra CARLA e Autoware [1], mediata dal ROS bridge [14]. In tale

configurazione, CARLA riproduce l'ambiente virtuale, la dinamica del veicolo e i sensori, mentre Autoware implementa la catena funzionale di percezione, pianificazione e controllo. Il ROS bridge abilita lo scambio in tempo reale dei dati sensoriali e dei comandi di guida, realizzando una simulazione *closed-loop* in cui le decisioni dell'ADS influenzano direttamente l'ambiente simulato e, viceversa, le condizioni ambientali aggiornate vengono immediatamente recepite dal sistema di guida.

Scelta del simulatore A differenza di CARLA, che dispone di una community ampia e attiva e riceve aggiornamenti periodici, LGSVL ha subito un rallentamento significativo a seguito della cessazione del supporto ufficiale da parte di LG nel 2022, che ne ha compromesso l'evoluzione e la manutenzione. Per queste ragioni, nell'analisi condotta in questa tesi è stato scelto CARLA come simulatore di riferimento.

2.4 Related works

Tra i contributi più rilevanti va citato il *Search-Based and Fuzz Testing* (SBFT) [15], nato come evoluzione del *Search-Based Software Testing* (SBST) per esplorare l'applicazione delle tecniche di *Search-Based Software Engineering* e del fuzzing al testing del software, con l'obiettivo di automatizzare il processo e promuovere nuove metodologie di valutazione. Negli ultimi anni il workshop ha esteso il proprio ambito ai sistemi di guida autonoma, istituendo una competizione dedicata in cui i tool di generazione di scenari vengono confrontati utilizzando il simulatore *BeamNG.tech*, all'interno di un'infrastruttura comune che gestisce l'integrazione dei generatori, valida e visualizza i test, esegue le simulazioni e produce report comparativi. In questo contesto, gli strumenti sono valutati sulla capacità di generare scenari validi ed efficaci nell'evidenziare difetti di guida quali uscite di strada, invasioni di corsia e arresti improvvisi.

Un ulteriore riferimento concettuale è sicuramente SafeBench [16], una piattaforma nata per fornire un ambiente unificato basato su CARLA per valutare le prestazioni degli agenti di guida autonoma.

Il presente lavoro si discosta da tali approcci poiché non valuta la capacità dei tool di generare topologie stradali, né quella degli agenti di reagire a scenari predefiniti. L'attenzione è invece rivolta alla capacità dei generatori di introdurre criticità e variabilità attraverso lo spawn degli attori, in condizioni ambientali differenti e con configurazioni eterogenee. In questo contesto, gli agenti non rappresentano l'oggetto della valutazione, ma solo un supporto per stimare l'efficacia degli scenari prodotti. Pur con queste differenze, sia l'impostazione competitiva di SBFT sia l'approccio di benchmarking multi-livello proposto da SafeBench sono stati assunti come riferimenti concettuali nella metodologia valutativa adottata in questo lavoro.

CAPITOLO 3

Metodologia

Questo capitolo descrive l'intero percorso metodologico seguito per l'analisi comparativa condotta in questo lavoro. A guidare lo sviluppo della metodologia sono state tre domande di ricerca, che hanno orientato l'intero processo rappresentando il filo conduttore del lavoro, influenzando sia la selezione dei tool da analizzare, sia la progettazione delle componenti impiegate per la raccolta e l'elaborazione dei dati. In apertura vengono presentati gli strumenti esplorati in letteratura e i criteri adottati per selezionare quelli più rilevanti rispetto agli obiettivi dello studio. Successivamente sono illustrate alcune scelte metodologiche, come il *system under stress*, e viene introdotta l'architettura per la raccolta dei dati: dal framework di logging sviluppato, con i relativi moduli di arricchimento, di analisi funzionale e dinamica, fino alla pipeline di clustering, impiegata per raggruppare gli scenari critici e consentire il calcolo delle metriche di diversità. Il capitolo si chiude con la definizione del quadro di valutazione, che organizza le metriche adottate nelle dimensioni di criticità, diversità ed efficacia.

3.1 Research Questions

L'analisi comparativa è stata guidata da tre domande di ricerca, che riflettono le dimensioni chiave identificate per la valutazione degli strumenti di generazione di scenari. Queste domande costituiscono il quadro di riferimento teorico che orienta le scelte metodologiche e anticipa l'analisi dei risultati presentata nel Capitolo 4.

Q RQ₁. *In che misura i diversi tool di generazione di scenari sono in grado di mettere sotto stress un sistema di guida autonoma, producendo condizioni critiche?*

Q RQ₂. *Quanto sono variegati gli scenari generati dai tool, e fino a che punto riescono a esplorare tipologie di situazioni differenti?*

Q RQ₃. *Qual è la capacità dei tool di concentrare la generazione su scenari effettivamente rilevanti per la validazione del sistema?*

Le risposte a queste domande saranno sviluppate nel capitolo successivo, sulla base dei dati raccolti e delle metriche definite nelle sezioni seguenti.

3.2 Tool analizzati

In questa sezione vengono dapprima descritti a livello introduttivo diversi strumenti di generazione di scenari, organizzati secondo la classificazione introdotta nella Sezione 2.3.1. Per ciascuna categoria sono presentati i tool più rappresentativi, mettendone in evidenza le caratteristiche distintive. In un secondo momento l'attenzione si concentra sugli strumenti selezionati in base ai criteri e agli obiettivi di questo lavoro, che vengono analizzati in modo più approfondito.

Knowledge-based Gli strumenti di questa classe condividono l'impiego di *Large Language Model* per tradurre descrizioni testuali in scenari, differenziandosi per le modalità adottate. **ChatScene** [17] e **ScenicNL** [18] generano scenari nel *Domain Specific Language (DSL)* Scenic, il primo tramite *retrieval* di frammenti da un dataset, il secondo con tecniche di prompting avanzate come il *Tree of Thought*. **TARGET** [19] codifica nel proprio DSL regole estratte dal codice della strada. **TTSG** [20] seleziona strade candidate con un ranking geometrico per mantenere coerenza tra geometria e

comportamenti descritti. **LEGEND** [21] adotta un approccio top-down a due fasi: un primo LLM estrae informazioni chiave dal testo, un secondo le traduce in scenari.

Adversarial Gli strumenti di questa categoria mirano a costruire scenari particolarmente critici per testare la robustezza dell'ADS. **KING** [22] ottimizza il traffico di sfondo con un modello cinematico semplificato, **AVASTRA** [23] addestra un agente avversario via *Reinforcement Learning*, mentre **DeepManeuver** [24] introduce perturbazioni visive per mettere sotto stress il modulo di percezione.

Data-driven Gli approcci basati su dati reali includono **DriveSceneGen** [25], che combina un modello a diffusione per ricostruire mappa e stato iniziale degli attori con un modello di previsione multimodale per generare l'evoluzione delle traiettorie.

Criteri di selezione Coerentemente con quanto discusso nel paragrafo *Scelta del simulatore*, sono stati considerati esclusivamente gli strumenti basati sul simulatore CARLA. Al fine di garantire confronti equi ed evitare bias dovuti a differenze tra versioni, il filtro è stato applicato anche sulla versione del simulatore, adottando la 0.9.13 come riferimento. All'interno di questo sottoinsieme, molti strumenti sono risultati essere ancora in fase di sviluppo o con documentazione parziale, ragion per cui non sono stati considerati per l'analisi comparativa effettuata in questo studio. A valle di questa scrematura, l'analisi si è concentrata su framework conformi ai criteri appena descritti, stabili ed effettivamente eseguibili. Sono stati dunque selezionati tre tool rappresentativi della classe *knowledge-based* e riconducibili al paradigma della *search-based generation*: **ScenarioFuzz-LLM** [26], **TM-Fuzzer** [27] e **SimADFuzz** [28].

Tool scelti

I tre strumenti scelti, seppur con differenze significative, condividono una stessa architettura di fondo: il *fuzz testing* [2] guidato da algoritmi genetici multi-obiettivo, in particolare il *NSGA-II*. Il funzionamento può essere visto come un ciclo evolutivo ispirato alla selezione naturale descritta da *Darwin*. Si parte da una popolazione iniziale di scenari (*seed pool*), che viene eseguita in simulazione. Ogni scenario è valutato tramite una funzione obiettivo che misura la capacità di mettere in difficoltà il veicolo ego. Gli scenari più "adatti" vengono selezionati e danno origine, tramite gli operatori genetici di *crossover* e *mutazione*, a nuove generazioni di scenari.

Rispetto a un algoritmo genetico classico, *NSGA-II* introduce tre innovazioni fondamentali:

- **Nondominated sorting:** suddivide la popolazione in livelli di soluzioni Pareto-ottimali, cioè non peggiori rispetto alle altre su tutti gli obiettivi considerati;
- **Crowding distance:** una misura della densità di soluzioni nello spazio degli obiettivi, utilizzata per mantenere varietà ed evitare che l'evoluzione converga troppo presto su poche soluzioni simili;
- **Elitismo:** assicura che le soluzioni migliori già trovate vengano preservate e trasmesse alle generazioni successive.

Il processo continua fino al raggiungimento di una *stopping condition*, che può essere dettata da un numero prefissato di iterazioni o da un budget temporale.

3.2.1 SimADFuzz

SimADFuzz è un framework che integra all'interno del proprio funzionamento una *feedback-guided generation*. Il cuore della valutazione è il *Violation Prediction Model* (VPM), un Transformer encoder che, anziché basarsi su valori aggregati, analizza l'intera sequenza temporale degli stati dei veicoli (posizione, velocità, accelerazione e direzione) e stima la probabilità che lo scenario conduca ad una violazione. Questo punteggio è combinato con altre tre metriche: la *distanza minima tra veicoli* (MDBV), il numero di *violazioni uniche* e lo *SDC-Scissor score*, una metrica che, tramite un *Random Forest*, valuta la complessità geometrica della strada (lunghezza, numero di curve). La

generazione di nuovi scenari segue logiche mirate: oltre alla *mutazione distance-guided*, che sostituisce NPC poco rilevanti, l'operatore di *crossover* scambia le traiettorie degli NPC solo quando queste presentano un'intersezione con la traiettoria seguita dal veicolo ego, così da massimizzare la possibilità di interazioni critiche.

3.2.2 TM-Fuzzer

TM-Fuzzer adotta un approccio peculiare, concepito per sfidare costantemente il veicolo ego durante la simulazione. Il nucleo del framework è il *Traffic Management Engine*, che gestisce dinamicamente l'inserimento e la rimozione dei veicoli NPC. A differenza di altri strumenti che fissano a priori i punti di spawn, gli NPC vengono collocati in posizioni topologicamente rilevanti (corsie parallele, incroci), così da garantire interazioni frequenti con il veicolo ego. Al fine di preservare il realismo ed attribuire la responsabilità al veicolo ego, ogni inserimento è preceduto da un controllo di sicurezza che esclude eventuali situazioni infattibili.

La varietà degli scenari è affidata al modulo denominato *Diversity Analysis Engine*, che analizza le traiettorie generate negli scenari critici. Queste vengono trasformate in immagini e processate da una rete neurale pre-addestrata (*ResNet-50*) per l'estrazione di feature che vengono poi raggruppate in dei cluster. Su questa base viene calcolato un indice chiamato *Similarity in Trajectories and Safety Violations (STSV)*, che combina più indicatori: la *distanza minima tra veicoli (MDBV)*, la *variazione di accelerazione (VOA)*, la *distanza di uno scenario dal centroide del cluster (SCD)* e la *numerosità del cluster stesso (CNC)*. In questo modo, il framework cerca di privilegiare scenari sia critici che diversi da quelli già esplorati, ampliando lo spettro delle violazioni individuabili.

3.2.3 ScenarioFuzz-LLM

ScenarioFuzz-LLM è un tool sviluppato da gran parte degli autori di *TM-Fuzzer* e ne eredita la struttura di base: un ciclo genetico con gestione dinamica del traffico di background. La differenza principale risiede nell'uso di un *Large Language Model (LLM)* per affrontare il problema della diversità degli scenari. Ad ogni iterazione, all'LLM viene passato esclusivamente il frame più critico dello scenario, ovvero quello in cui la distanza minima tra veicoli risulta più bassa. Questo *snapshot* viene

rielaborato dal modello in una descrizione testuale sintetica che evidenzia i veicoli più critici, la tipologia e la potenziale severità dell'incidente nella situazione considerata. Le descrizioni così ottenute alimentano una libreria di scenari che consente un confronto efficiente tra gli scenari generati.

La valutazione evolutiva si basa su tre metriche: la *distanza minima tra veicoli* (MDBV), la *variazione di accelerazione* (VOA) e un indice chiamato *Comprehensive Inter-Scenario Variability* (CISV), calcolato dall'LLM confrontando lo scenario corrente con quelli presenti nella libreria.

Per evitare convergenza prematura dell'evoluzione genetica, il framework introduce tre controlli di stabilità (sul fronte di Pareto, sulla crowding distance e sulla varianza degli obiettivi). Quando questi indicatori segnalano stagnazione, viene attivata la *mutazione guidata da LLM*, che propone modifiche mirate ai parametri degli NPC (ad esempio posizione e velocità) al fine di generare nuove interazioni e stimolare la diversificazione degli scenari.

3.3 Raccolta dei dati

La fase di raccolta dei dati consiste nell'esecuzione sistematica degli scenari generati dai diversi framework scelti in condizioni uniformi, così da garantire la confrontabilità dei risultati. Ogni simulazione è monitorata e registrata tramite un sistema di logging dedicato, in grado di tracciare sia dati statici che dinamici, che costituiranno la base per il calcolo delle metriche, consentendo così l'effettivo confronto.

3.3.1 System under stress: BehaviorAgent

Per valutare i diversi framework generativi, è stato adottato come *system under stress* il *BehaviorAgent*¹ fornito nativamente da CARLA. Si tratta di un agente di guida basato su regole, pienamente integrato nel simulatore, che rappresenta un ADS funzionale ma non avanzato. All'interno del suo modulo di pianificazione, l'agente utilizza l'algoritmo A^* per il calcolo del percorso globale: una volta specificata la posizione iniziale e quella di destinazione, viene generato il cammino più breve che l'agente segue fino al completamento della missione, garantendo un comportamento deterministico e ripetibile.

La scelta di questo agente è stata guidata da due considerazioni principali. In primo luogo, l'uso del BehaviorAgent consente un confronto equo tra framework eterogenei, alcuni dei quali prevedono integrazioni con ADS diversi. In secondo luogo, la disponibilità nativa in CARLA e il discreto utilizzo in letteratura [29, 30] come *baseline* rendono questa scelta sensata e metodologicamente solida.

In questo lavoro, il BehaviorAgent è stato configurato nella modalità *cautious*, mettendo meglio in evidenza le criticità introdotte dagli scenari. In aggiunta, l'agente è stato esteso in maniera euristica per garantire il rispetto dei segnali di stop, introducendo una componente di controllo dedicata (*StopManager*) integrata nel ciclo decisionale dell'agente.

¹https://carla.readthedocs.io/en/0.9.13/adv_agents/, accessed on 24/09/2025

3.3.2 Sistema di logging

Per consentire il calcolo delle metriche e, quindi, l'effettiva valutazione dei tool è stato sviluppato un sistema di logging dedicato. Il nucleo è costituito dal modulo `CarlaBasicLogger`, incaricato di produrre un log strutturato in formato JSON, organizzato nelle seguenti sezioni principali:

- **metadati**: informazioni generali relative al tool di generazione, identificativo dello scenario, mappa utilizzata, tempo di avvio e parametri della simulazione;
- **missione del veicolo ego**: posizione di spawn, destinazione pianificata e sequenza dei *waypoint* calcolati dal *planner*;
- **attori dinamici**: elenco degli NPC con tipo, ruolo e coordinate di spawn;
- **eventi principali**: registro di collisioni, attraversamenti col semaforo rosso, superamenti del limite di velocità e violazioni dello stop;
- **frames**: stato dettagliato del veicolo ego e degli altri attori dinamici per ogni tick del simulatore, comprensivo di coordinate, velocità e vertici delle bounding box.

A supporto del logger è stato implementato un ulteriore modulo denominato `ViolationMonitor`, che rileva in tempo reale le principali violazioni comportamentali del veicolo ego:

- **speeding**: superamento del limite di velocità con margine di tolleranza e periodo di grazia dopo il cambio limite;
- **red light violation**: attraversamento di un incrocio durante il semaforo rosso;
- **stop sign violation**: logica euristica che verifica la fermata in corrispondenza di uno *stop sign*.

L'output del logger rappresenta l'insieme grezzo dei dati raccolti durante la simulazione e costituisce la base uniforme per le analisi successive. In parallelo viene inoltre salvato un file binario `.rec`, generato tramite il recorder nativo di

CARLA², che consente di riprodurre fedelmente l'intera esecuzione, inclusi spawn e distruzione degli attori, stati dei semafori e dinamiche veicolari.

3.3.3 Arricchimento dei log

Il file prodotto dal logger, pur contenendo informazioni complete sugli attori, sulla missione del veicolo ego e sugli eventi registrati, non è immediatamente adatto a un confronto sistematico tra i tool. Per questo è stata introdotta una fase di *arricchimento*, implementata tramite tre moduli indipendenti, che trasformano i log grezzi in strutture arricchite da un primo insieme di metriche, successivamente aggregabili.

Analisi della criticità Il modulo `CriticalAnalyzer` elabora i log *frame-by-frame*, ricostruisce la geometria delle bounding box e calcola distanze e velocità relative. Da queste elaborazioni derivano tre indicatori fondamentali: distanza minima tra attori, tempo minimo alla collisione e tempo di esposizione al rischio, che delineano il livello di pericolosità degli scenari.

Analisi funzionale Il modulo `FunctionalAnalyzer` valuta le prestazioni funzionali del veicolo ego rispetto alla missione pianificata. Il percorso registrato viene confrontato con la rotta teorica pianificata dal *planner*, stimando il tasso di completamento della missione, il tempo di completamento, le deviazioni laterali e la distanza effettivamente percorsa. Queste misure distinguono tra scenari conclusi e scenari interrotti e permettono di valutare la stabilità nell'esecuzione della missione.

Analisi cinematica Il modulo `DynamicAnalyzer` analizza la dinamica longitudinale del veicolo. A partire dai profili di velocità e accelerazione, calcola indicatori statistici come velocità media e massima, accelerazioni medie, percentili e valori di picco, evidenziando eventuali episodi di guida aggressiva e descrivendo il comportamento complessivo del veicolo ego.

²https://carla.readthedocs.io/en/0.9.13/adv_recorder/, accessed on 24/09/2025

3.3.4 Pipeline di clustering

Per analizzare la diversità degli scenari generati dai vari tool è stata definita una pipeline di clustering. Sono stati considerati i log arricchiti degli scenari che hanno prodotto collisioni o violazioni, da cui è stato ricavato per ciascuno un vettore di feature contenente indicatori di criticità, dinamiche di guida ed eventi registrati.

Pre-processing I dati sono stati sottoposti a una fase di preparazione volta a renderli confrontabili e più adatti al clustering. In particolare, è stata effettuata una normalizzazione tramite il *RobustScaler* per ridurre l’impatto degli outlier e successivamente una trasformazione di riduzione della dimensionalità mediante *UMAP* riprendendo l’approccio di SBFT [15], così da ottenere una rappresentazione più compatta ed efficiente dello spazio delle feature.

Clustering e scelta del k Il raggruppamento è stato eseguito con l’algoritmo *KMeans*. La determinazione del numero ottimale di cluster si è basata su due criteri complementari: l’indice di *silhouette*, che valuta coesione e separazione dei cluster con un punteggio compreso tra -1 e 1 , e il metodo dell’*elbow point*, che individua il punto di flesso nella curva del *Within-Cluster Sum of Squares* (WCSS), ossia il momento in cui l’aggiunta di ulteriori cluster non produce più un miglioramento significativo nella compattezza interna. La regola adottata è la seguente: quando i due criteri coincidono si assume quel valore di k ; in caso di discrepanza, viene selezionato il valore suggerito dall’indice di *silhouette* solo se il relativo punteggio supera di almeno 0.1 quello dell’*elbow*, altrimenti si mantiene la scelta di quest’ultimo. L’impiego dell’algoritmo *KMeans* con la selezione del numero di cluster basata sull’indice di *silhouette* segue l’impostazione di SBFT [15], che privilegia un criterio empirico piuttosto che l’uso di valori arbitrari (*magic numbers*) per la scelta di k . La combinazione con il metodo dell’*elbow point* costituisce invece un arricchimento metodologico introdotto in questo lavoro.

3.3.5 Integrazione del framework nei tool selezionati

I framework analizzati non erano originariamente progettati per produrre log completi e pienamente coerenti con gli obiettivi di questo lavoro. Per questo motivo, il loro codice è stato esteso mediante l’inserimento di punti di intercettazione nelle principali fasi di esecuzione. Ciò ha reso possibile raccogliere in modo trasparente informazioni su missione, attori dinamici ed eventi critici, instradandole verso il sistema di logging precedentemente descritto. È importante sottolineare che tali modifiche hanno riguardato esclusivamente l’integrazione e il logging, senza alterare la logica di generazione dei framework, rimasta invariata rispetto all’implementazione originale.

Nel caso di *ScenarioFuzz-LLM*, oltre a queste modifiche comuni a tutti i tool, è stata introdotta la sostituzione del *Large Language Model* impiegato come guida alla mutazione: al posto di *GPT-4-turbo* è stato utilizzato *Gemini-2.5-pro*, mantenendo inalterata la logica di prompting e di parsing. Questa scelta è stata dettata esclusivamente da motivazioni pratiche, legate alla disponibilità di crediti gratuiti offerti dalla piattaforma *Google Vertex AI* ³, e non modifica in alcun modo la logica generativa del framework.

Ambiente di esecuzione e configurazione sperimentale. Le simulazioni sono state eseguite su una macchina con sistema operativo *Ubuntu 22.04*, processore *AMD Ryzen 5 7500F*, 16GB di RAM e GPU *AMD Radeon RX 7700 XT*. L’ambiente software è stato configurato localmente, evitando l’impiego di containerizzazione tramite Docker a causa di problematiche di compatibilità con GPU AMD. Per garantire condizioni di confronto eque, ciascun tool è stato eseguito con un budget temporale uniforme pari a **3 ore di simulazione**, in linea con l’approccio adottato anche dalla competizione SBFT [15]. Questa scelta ha permesso di uniformare le risorse computazionali dedicate ed evitare bias dovuti a configurazioni eterogenee, concentrando l’analisi esclusivamente sul comportamento generativo dei tool.

³<https://cloud.google.com/vertex-ai>

3.4 Metodologia di valutazione

La valutazione dei tool di generazione di scenari richiede una metodologia chiara e strutturata, capace di cogliere non solo la quantità, ma anche la qualità e la rilevanza degli scenari prodotti. Un numero elevato di scenari non è infatti sufficiente: ciò che rende utile uno strumento è la capacità di proporre scenari significativi e rilevanti. A tal fine, in questa sezione vengono illustrati i criteri definiti per analizzare e confrontare i diversi strumenti considerati.

L'attenzione si concentra su tre dimensioni complementari:

- **criticità**, intesa come il potenziale degli scenari di mettere l'ADS sotto stress in condizioni di rischio o fallimento;
- **diversità**, intesa come la varietà delle situazioni generate e la capacità di differenziarle, evitando ridondanze e favorendo una copertura bilanciata delle possibili interazioni;
- **efficacia**, intesa come la frequenza con cui il tool genera scenari effettivamente critici.

3.4.1 Metriche e punteggi

Per rendere oggettiva la valutazione è stato definito un set di metriche che quantificano le proprietà più rilevanti degli scenari generati.

Criticità

La criticità è valutata secondo un approccio ispirato all'*overall score* di *Safebench* [16], basato sulla combinazione pesata e normalizzata delle metriche descritte di seguito. In *SafeBench* collisioni e infrazioni sono trattate come penalità per l'agente, poiché valori più bassi indicano migliori prestazioni. Nel presente lavoro, invece, esse sono considerate indicatori positivi di criticità, in quanto segnalano scenari realmente sfidanti per il sistema sotto test (SUT).

Metriche di criticità Le metriche si dividono in due categorie: *indicatori diretti di rischio*, che includono collisioni e violazioni del codice della strada e descrivono

eventi critici osservabili, e misure *funzionali*, riconducibili al livello *Functionality* di *SafeBench*, che non esprimono rischi immediati ma indicano la difficoltà intrinseca dello scenario.

- **Collision Rate (CR):** frequenza con cui il veicolo ego è coinvolto in collisioni.
- **Traffic Rule Violation Rate (TRVR):** frequenza di violazioni del codice della strada (semaforo rosso, stop, limiti di velocità).
- **Completion (Comp):** percentuale del percorso pianificato completata.
- **Route Following Stability (RF):** capacità del veicolo di mantenersi vicino alla rotta pianificata.
- **Time Spent (TS):** tempo medio necessario al completamento della missione, calcolato solo sugli scenari completati al 100%.

Punteggio di criticità Il punteggio complessivo $C(T)$ è ottenuto come somma pesata delle metriche normalizzate:

$$C(T) = \sum_{i=1}^n w_i \cdot M_i^*(T)$$

dove w_i è il peso della i -esima metrica e $M_i^*(T)$ il valore normalizzato di $M_i(T)$ rispetto al massimo M_i^{max} .

$$M_i^*(T) = \begin{cases} \frac{M_i(T)}{M_i^{max}}, & \text{se la criticità è direttamente proporzionale alla metrica} \\ 1 - \frac{M_i(T)}{M_i^{max}}, & \text{altrimenti} \end{cases}$$

Il punteggio $C(T)$ assume valori compresi tra 0 e 1: valori prossimi a 1 indicano che il tool ha generato scenari altamente critici, mentre valori vicini a 0 corrispondono a scenari poco critici.

L'orientamento, i valori massimali e i pesi sono riportati nella Tabella 3.1.

Analisi del rischio latente Oltre agli esiti critici, è utile considerare condizioni di *near-miss*, cioè situazioni in cui l'ego veicolo si avvicina a limiti di sicurezza senza però produrre un fallimento. Questo consente di considerare scenari che, pur senza collisioni, mettono l'ADS in condizioni potenzialmente critiche.

Metriche di prossimità

- **Time-to-Collision (TTC)**: tempo residuo alla collisione, assumendo velocità e traiettorie costanti.
- **Time-Exposed Time-to-Collision (TET)**: durata complessiva in cui il TTC resta sotto una soglia critica.
- **Minimum Distance Between Vehicles (MDBV)**: minima distanza osservata tra l'ego e altri veicoli.

Valutazione Per il rischio latente non viene definito un punteggio aggregato. L'analisi è comparativa, basata sulle medie delle metriche considerate, e si applica solo agli scenari senza collisioni.

Diversità

Una validazione efficace non può basarsi su scenari ripetitivi, ma richiede una copertura ampia e bilanciata delle possibili interazioni. Per questo la diversità viene analizzata tramite tecniche di clustering, che permettono di raggruppare gli scenari in tipologie omogenee e di valutarne la distribuzione complessiva, come proposto da SBFT [15]. L'analisi è condotta sul solo sottoinsieme di scenari che hanno prodotto una condizione di *failure*, intesa come collisione o violazione, così da focalizzare la valutazione sui casi realmente significativi.

Metriche di diversità

- **Cluster Coverage (Ccov)**: misura la capacità di esplorare tipologie di scenari eterogenee. Dato l'insieme dei cluster $C = \{c_1, \dots, c_{K^*}\}$, la copertura è definita come

$$C_{\text{cov}}(T) = \frac{1}{K^*} \cdot \sum_{i=1}^{K^*} \lambda_i(T)$$

con

$$\lambda_i(T) = \begin{cases} 1 & \text{se il tool } T \text{ ha almeno uno scenario nel cluster } i \\ 0 & \text{altrimenti} \end{cases}$$

- **Cluster Entropy (Cent):** valuta la distribuzione degli scenari tra i cluster. Dato un tool T , con $p_i(T)$ probabilità che uno scenario appartenga al cluster c_i , l'entropia normalizzata è

$$C_{\text{ent}}(T) = -\frac{1}{\log_2 K^*} \cdot \sum_{i=1}^{K^*} p_i(T) \cdot \log_2 p_i(T)$$

Punteggio di diversità Il punteggio complessivo $D(T)$ è la combinazione pesata di *coverage* ed *entropy*:

$$D(T) = 0.5 \cdot C_{\text{cov}}(T) + 0.5 \cdot C_{\text{ent}}(T)$$

Il valore di $D(T)$ varia tra 0 e 1: valori prossimi a 1 indicano scenari eterogenei e bilanciati, mentre valori bassi segnalano una generazione ridondante e ristretta a poche tipologie.

Efficacia

L'efficacia misura la capacità di un tool di generare scenari critici. È definita come il rapporto tra il numero di scenari che hanno prodotto collisioni o violazioni e il numero totale di scenari generati. Un valore elevato, prossimo a 1, indica che il tool concentra la generazione su casi di interesse, riducendo scenari poco rilevanti e ottimizzando l'uso delle risorse computazionali.

Tabella 3.1: Metriche, valori massimali M_{max} e pesi adottati nel calcolo di $C(T)$.

Metrica	M_{max}	Orient.	Peso w_i
Collision Rate (CR)	1	↑	0.50
Traffic Rule Violation Rate (TRVR)	1	↑	0.30
Completion (Comp)	1	↓	0.10
Route Following (RF)	1	↓	0.05
Time Spent (TS)	120 s	↑	0.05
Totale			1.00

↑: valori elevati indicano maggiore criticità. ↓: valori ridotti indicano maggiore criticità.

Tabella 3.2: Sintesi delle metriche considerate e significato dei valori.

Metrica	Range	Interpretazione dei valori
Collision Rate (CR)	[0, 1]	Valori alti indicano una frequenza maggiore di collisioni e quindi una criticità più elevata.
Traffic Rule Violation Rate (TRVR)	[0, 1]	Valori alti segnalano più violazioni delle regole del traffico e maggiore criticità.
Completion (Comp)	[0, 1]	Valori bassi indicano missioni interrotte e quindi maggiore criticità.
Route Following (RF)	[0, 1]	Valori bassi riflettono deviazioni più ampie dalla rotta pianificata e minore stabilità.
Time Spent (TS)	> 0 s	Valori alti corrispondono a scenari più lunghi e complessi.
Time-to-Collision (TTC)	> 0 s	Valori bassi indicano un avvicinamento rapido a una situazione di rischio.
Time-Exposed TTC (TET)	> 0 s	Valori alti denotano un'esposizione prolungata a condizioni critiche.
Minimum Distance (MDBV)	> 0 m	Valori bassi rappresentano una distanza ridotta dagli altri veicoli e quindi un rischio maggiore.
Cluster Coverage (Ccov)	[0, 1]	Valori alti segnalano che vengono esplorate più tipologie di scenari.
Cluster Entropy (Cent)	[0, 1]	Valori alti indicano una distribuzione più bilanciata degli scenari tra i cluster.

CAPITOLO 4

Analisi dei risultati

In questo capitolo vengono discussi i risultati dell'applicazione della metodologia descritta nel Capitolo 3. L'analisi comparativa riguarda i tre tool *ScenarioFuzz-LLM*, *SimADFuzz* e *TM-Fuzzer*, con l'obiettivo di evidenziarne punti di forza e limiti rispetto a criticità, diversità ed efficacia.

La presentazione segue un andamento progressivo: si parte dai risultati aggregati per ciascuna dimensione, si entra poi nel dettaglio delle singole metriche e infine si propone una sintesi comparativa, utile a delineare i diversi profili strategici emersi. In chiusura, i risultati vengono ricondotti alle research questions, fornendo le risposte emerse dall'analisi empirica.

4.1 Valutazione della criticità

La criticità misura la capacità di un tool di generare scenari che stressano l'ADS. L'analisi si articola in due parti: il profilo complessivo basato su esiti diretti e indiretti, e lo studio del rischio latente (*near-miss*).

4.1.1 Profilo di criticità e punteggio aggregato

La Tabella 4.1 riporta i valori medi e il punteggio aggregato $C(T)$, mentre la figura 4.1 mostra il profilo comparativo.

Dall'analisi delle metriche emergono tre profili distinti:

- **ScenarioFuzz-LLM**: concentra la criticità nella frequenza di collisioni e violazioni, entrambe molto elevate. A questo si accompagna però un livello di completamento e stabilità elevato, segnale che gli scenari non compromettono la missione nella sua globalità, ma ne aumentano la densità di eventi rischiosi. Risulta quindi efficace nel sollecitare il sistema con episodi brevi ma ad alta intensità di pericolo.
- **SimADFuzz**: presenta la criticità diretta più bassa, con collisioni e violazioni ridotte al minimo. Compensando questa scarsità di eventi estremi, gli scenari risultano però molto lunghi e articolati (oltre 115 s in media), favorendo la valutazione della robustezza funzionale e della tenuta del sistema in condizioni di esposizione prolungata. È quindi meno adatto a generare fallimenti espliciti, ma più utile a osservare il comportamento dell'ADS su traiettorie estese.
- **TM-Fuzzer**: si colloca in una posizione intermedia, con metriche distribuite in modo più uniforme. I livelli di rischio sono moderati e la durata resta consistente, delineando un profilo equilibrato che consente di generare scenari vari senza spingere in maniera marcata né sul fallimento esplicito né sulla robustezza di lungo periodo.

Tabella 4.1: Risultati metriche di criticità

Tool	CR (%)	VR (%)	Comp (%)	RF (%)	TS (s)	C(T)
ScenarioFuzz-LLM	31.80	54.50	84.70	95.70	94.20	0.38
TM-Fuzzer	28.70	29.90	78.50	95.90	93.90	0.30
SimADFuzz	11.60	3.20	73.60	92.80	115.10	0.14

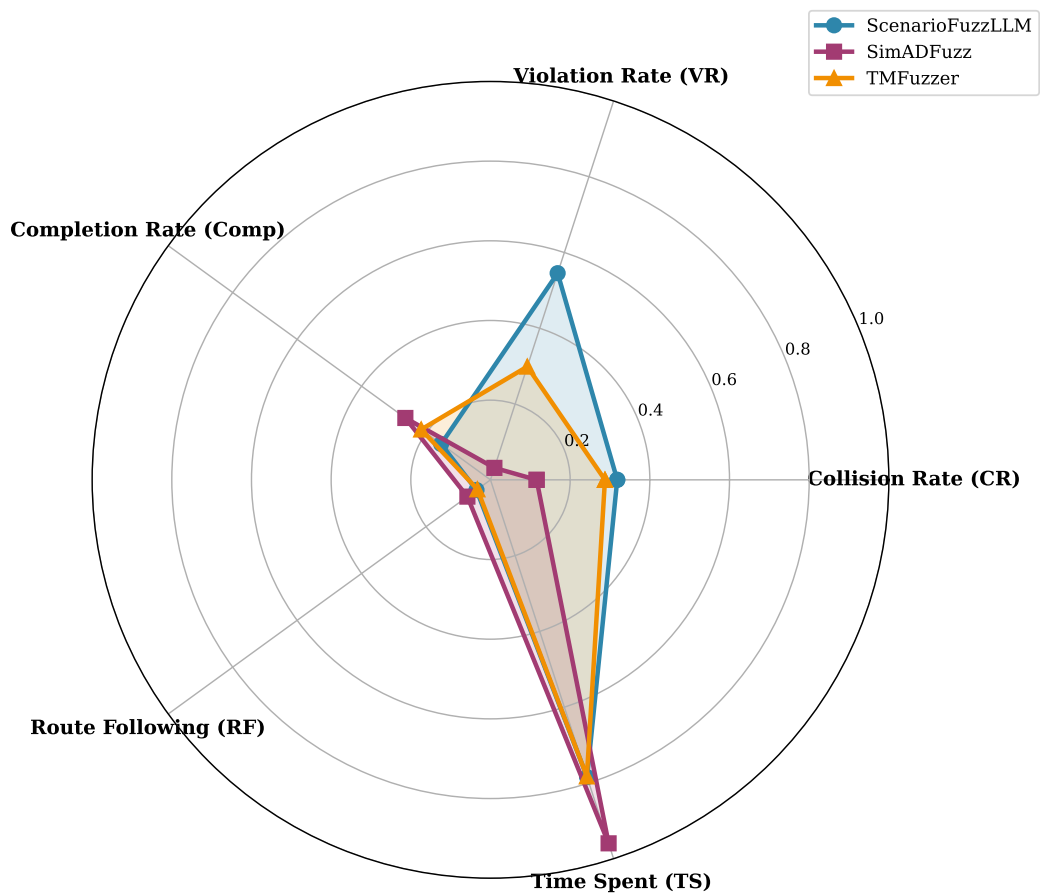
Profilo di Criticità Comparativo dei Tool

Figura 4.1: Profilo di criticità comparativo dei tool. Gli assi rappresentano le cinque metriche normalizzate. Un'area più ampia del poligono corrisponde a una maggiore criticità complessiva.

4.1.2 Analisi del rischio latente

Per gli scenari senza collisione sono state analizzate tre metriche: MDBV, TTC e TET. I boxplot di Figura 4.2 e le statistiche in Tabella 4.2 ne mostrano le distribuzioni.

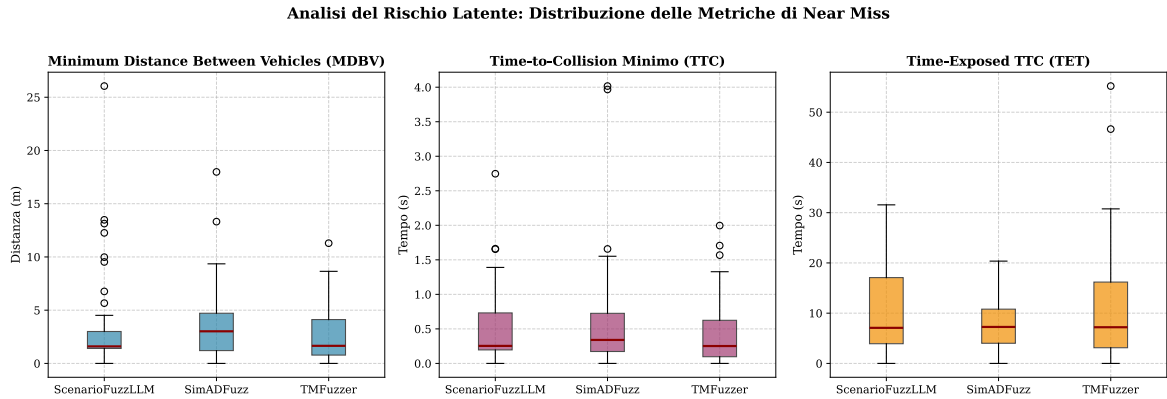


Figura 4.2: Distribuzione delle metriche di rischio latente per gli scenari senza collisione. Per MDBV e TTC, valori più bassi indicano un rischio maggiore. Per TET, valori più alti indicano un rischio più prolungato.

MDBV. ScenarioFuzz-LLM e TM-Fuzzer registrano distanze minime simili (mediane intorno a 1.6 m), sensibilmente inferiori a quelle di SimADFuzz (3.02 m). Ciò indica una maggiore capacità di generare situazioni in cui il veicolo ego si trova a distanze ravvicinate dai *background vehicles*, forzando condizioni di rischio più immediate. Si osservano inoltre outlier elevati in ScenarioFuzz-LLM (oltre 25 m), che ampliano la dispersione della distribuzione.

TTC. ScenarioFuzz-LLM e TM-Fuzzer presentano mediane identiche (0.25 s), segnalando una maggiore frequenza di situazioni di pericolo imminente. SimADFuzz mostra valori leggermente più alti (mediana 0.34 s) e un'estensione massima fino a 4 s, indicando una distribuzione meno aggressiva.

TET. Le mediane sono simili (circa 7 s) per tutti i tool, ma le differenze emergono nella variabilità. SimADFuzz mostra un intervallo interquartile ridotto, con valori concentrati attorno alla mediana e massimi limitati a circa 20 s. Al contrario, ScenarioFuzz-LLM e TM-Fuzzer presentano distribuzioni più ampie, con TM-Fuzzer

Tabella 4.2: Statistiche descrittive del rischio latente

Metrica	Tool	Mediana	IQR (Q3-Q1)	Min	Max
MDBV (m)	ScenarioFuzz-LLM	1.60	1.60	0.00	26.05
	SimADFuzz	3.02	3.50	0.00	17.99
	TM-Fuzzer	1.65	3.34	0.00	11.29
TTC (s)	ScenarioFuzz-LLM	0.25	0.53	0.00	2.75
	SimADFuzz	0.34	0.56	0.00	4.02
	TM-Fuzzer	0.25	0.52	0.00	2.00
TET (s)	ScenarioFuzz-LLM	7.08	13.16	0.00	31.56
	SimADFuzz	7.26	6.79	0.00	20.36
	TM-Fuzzer	7.20	13.07	0.00	55.20

che raggiunge valori estremi fino a 55 s. Ciò suggerisce che SimADFuzz tende a generare scenari più omogenei, mentre gli altri due includono episodi più eterogenei e talvolta molto prolungati.

🔗 **Answer to RQ₁.** ScenarioFuzz-LLM emerge come il tool più orientato alla generazione di scenari critici, TM-Fuzzer mostra un profilo bilanciato e variegato, mentre SimADFuzz privilegia complessità funzionale con rischi meno estremi ma più consistenti.

4.2 Valutazione della diversità

La diversità rappresenta la varietà delle tipologie di fallimento considerate, prendendo in esame esclusivamente scenari con collisioni o violazioni. La Tabella 4.3 riporta i punteggi aggregati $D(T)$ per ciascun tool.

Tabella 4.3: Sintesi delle metriche di diversità e del punteggio aggregato $D(T)$ per ciascun tool.

Tool	Cluster Coverage (Ccov)	Cluster Entropy (Cent)	$D(T)$
TM-Fuzzer	1.00	0.945	0.973
ScenarioFuzz-LLM	1.00	0.929	0.965
SimADFuzz	0.50	0.470	0.485

TM-Fuzzer e ScenarioFuzz-LLM raggiungono valori prossimi a quelli ideali, con copertura completa dei cluster ($Ccov = 1.00$) e alta entropia (rispettivamente 0.945 e 0.929). Questo significa che entrambi i tool riescono non solo a generare tutte le tipologie individuate, ma anche a distribuirle in modo relativamente bilanciato. SimADFuzz, al contrario, ottiene valori molto più bassi (copertura pari a 0.50 ed entropia 0.47): la sua generazione si concentra infatti su una porzione ristretta dello spazio dei fallimenti, trascurando intere tipologie e riducendo quindi la varietà complessiva degli scenari prodotti.

4.2.1 Analisi dei cluster

Per interpretare i punteggi di diversità è stato prima necessario determinare il numero ottimale di cluster. L'analisi combinata dell'indice di silhouette e della curva WCSS (elbow method) ha indicato come valore più appropriato $k = 4$, come mostrato in Figura 4.3. Il coefficiente di silhouette complessivo pari a 0.54 conferma la buona qualità della partizione.

Una volta fissato $k = 4$, la proiezione bidimensionale (Figura 4.4a) evidenzia la buona separazione dei gruppi, mentre la heatmap di Figura 4.4b mostra come ciascun tool abbia distribuito i propri scenari al loro interno.

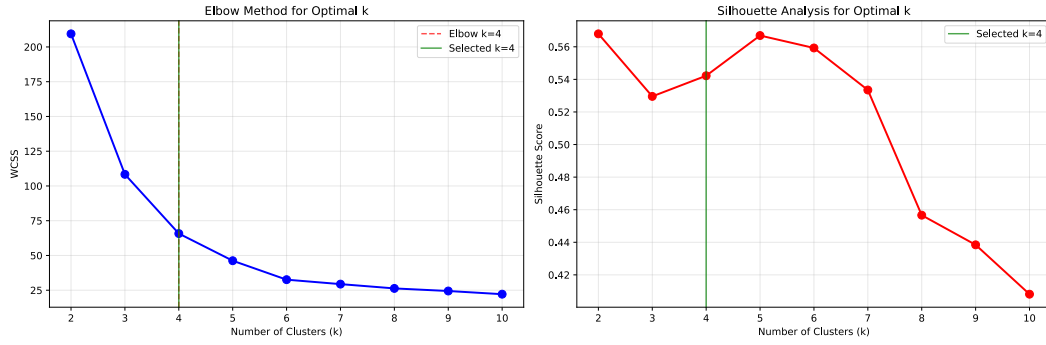
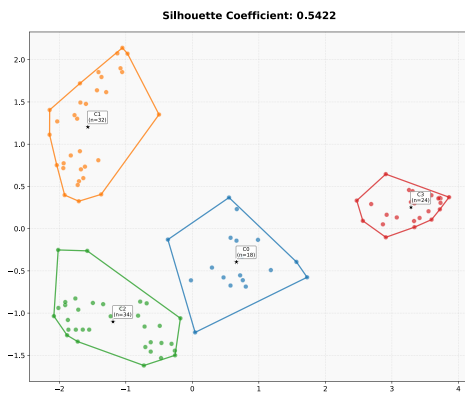
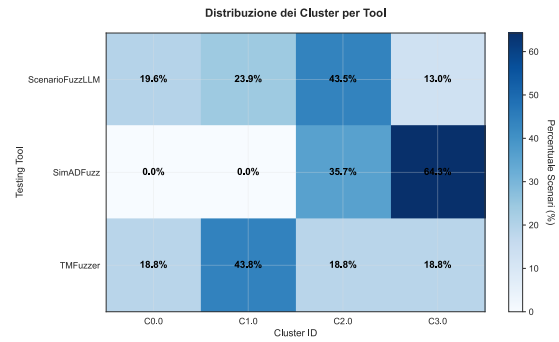


Figura 4.3: Confronto tra indice di silhouette e curva WCSS. Entrambi i metodi convergono su $k = 4$ come numero ottimale di cluster.



(a) Cluster degli scenari



(b) Distribuzione degli scenari per tool nei cluster

Figura 4.4: Cluster degli scenari ($k = 4$) e distribuzione dei tool nei cluster.

La distribuzione per tool evidenzia differenze marcate. ScenarioFuzz-LLM copre tutti i cluster, ma con percentuali variabili tra il 13% e il 43.5%, ottenendo un'entropia leggermente più bassa. TM-Fuzzer, pur concentrando il 43.8% in C1, mantiene valori quasi identici negli altri tre cluster (18.8%), raggiungendo così l'entropia più alta. SimADFuzz si limita a due cluster (64.3% in C3 e 35.7% in C2), lasciando vuoti C0 e C1 e riducendo drasticamente copertura ed entropia.

🔗 **Answer to RQ₂.** TM-Fuzzer e ScenarioFuzz-LLM emergono come generatori esplorativi, in grado di coprire tutti i cluster e distribuire i fallimenti in modo relativamente bilanciato. SimADFuzz, al contrario, mostra un approccio specializzato, concentrando i propri scenari in pochi cluster e riducendo la varietà delle tipologie di fallimento.

4.3 Valutazione dell'efficacia

L'efficacia misura la quota di scenari che portano a un fallimento. I punteggi di efficacia $E(T)$, riportati in Figura 4.5, offrono una sintesi immediata delle prestazioni dei tool e costituiscono la base per rispondere alla terza *research question*.

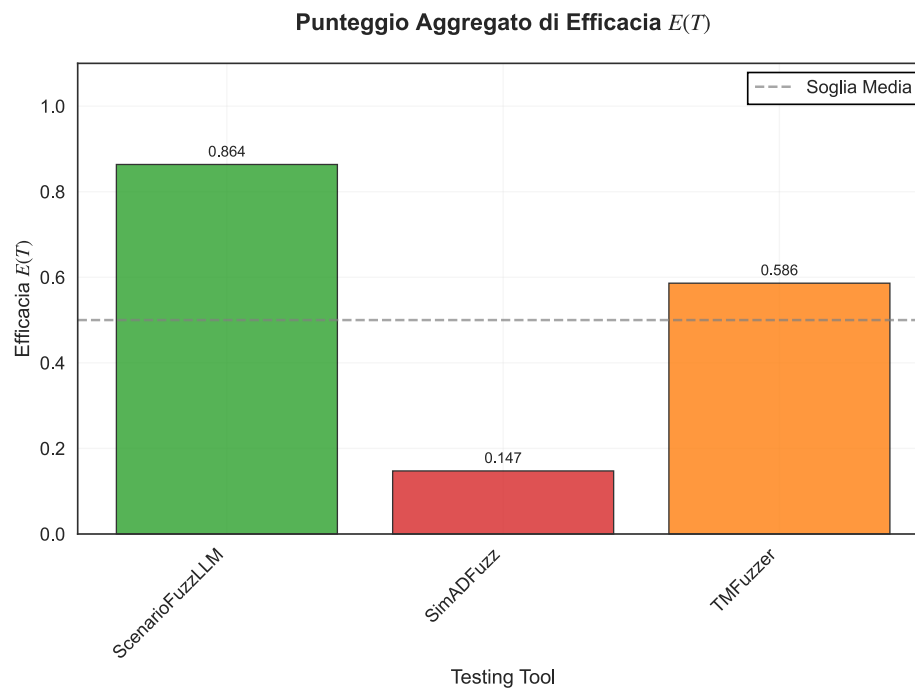


Figura 4.5: Punteggio aggregato di efficacia $E(T)$ per ciascun tool

🔗 **Answer to RQ₃.** ScenarioFuzz-LLM raggiunge l'efficacia più alta, con quasi nove scenari su dieci che portano a un fallimento. TM-Fuzzer si posiziona su valori solidi, dimostrando un buon equilibrio tra frequenza di fallimenti e diversità esplorata. SimADFuzz registra un'efficacia molto più bassa, riflettendo la sua tendenza a generare scenari lunghi e complessi piuttosto che situazioni di fallimento dirette.

Sintesi

L'analisi comparativa ha messo in luce tre approcci distinti. ScenarioFuzz-LLM privilegia la massimizzazione dei fallimenti diretti, con elevata criticità e ampia copertura di tipologie. TM-Fuzzer si conferma il profilo più equilibrato, con valori solidi di efficacia e diversità, pur senza estremizzare la criticità. SimADFuzz si caratterizza invece per scenari lunghi e articolati, che riducono l'incidenza di fallimenti espliciti ma consentono di osservare near-miss e robustezza funzionale.

A completamento, la Figura 4.6 mostra la distribuzione complessiva degli outcome, fornendo una sintesi visiva dei tre profili precedentemente descritti.

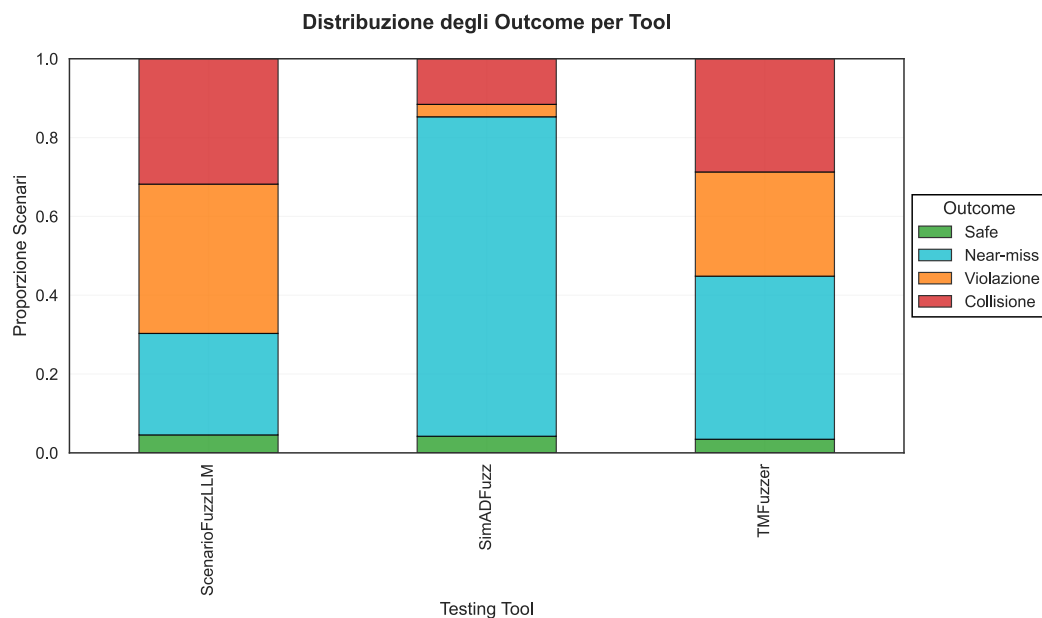


Figura 4.6: Distribuzione complessiva degli outcome

Conclusioni e Lavori Futuri

Sintesi dei risultati L'analisi condotta ha permesso di confrontare in maniera sistematica tre generatori di scenari basati su CARLA, mettendo in evidenza come ciascuno di essi esprima un profilo distinto. *ScenarioFuzz-LLM* ha mostrato la maggiore propensione a generare collisioni e violazioni esplicite, *TM-Fuzzer* si è distinto per l'equilibrio tra diversità ed efficacia, mentre *SimADFuzz* ha prodotto scenari lunghi e consistenti, più adatti a osservare la tenuta funzionale dell'agente nel tempo. Questi risultati suggeriscono che non esista uno strumento superiore in assoluto, ma piuttosto approcci complementari, utili a seconda degli obiettivi della validazione.

Limiti della sperimentazione La sperimentazione si è basata sul *BehaviorAgent* come sistema sotto stress, una scelta funzionale a garantire ripetibilità e semplicità di integrazione, ma inevitabilmente limitata nella rappresentazione della complessità degli ADS reali. Questo aspetto riduce la possibilità di generalizzare i risultati a sistemi più avanzati, che adottano pipeline reali di percezione, pianificazione e controllo. Un ulteriore aspetto da considerare riguarda la presenza di valori estremi nelle dinamiche veicolari, con accelerazioni longitudinali registrate che vanno oltre i 200 m/s^2 , del tutto irraggiungibili per un veicolo reale e riconducibili a disconti-

nuità numeriche del simulatore. Questi outlier mostrano come la simulazione possa produrre situazioni fisicamente implausibili.

Limiti legati alla natura dei tool analizzati Un ulteriore limite del presente studio riguarda la natura intrinseca dei framework considerati, tutti basati su algoritmi genetici. L’approccio evolutivo consente di esplorare spazi di ricerca ampi ed eterogenei, ma non garantisce un controllo fine e deterministico sui parametri degli scenari. La generazione rimane quindi guidata in modo statistico dalla funzione di fitness, senza possibilità di imporre condizioni specifiche o di riprodurre con precisione determinati comportamenti. Se da un lato questo livello di automazione rende possibile anche a persone non specializzate nella progettazione di scenari sottoporre un ADS a test significativi, dall’altro la randomicità e la scarsa controllabilità introdotte rimangono fattori da ponderare.

Prospettive future L’analisi ha mostrato come i tool selezionati offrano contributi complementari, ma anche limiti strutturali che aprono a diversi sviluppi futuri. Le linee di ricerca più promettenti potrebbero riguardare l’integrazione di ADS completi, l’ampliamento del ruolo degli LLM tramite l’arricchimento delle metriche di diversità e l’adozione di indicatori di realismo.

Integrazione di ADS completi Un primo sviluppo consiste nel sostituire il *BehaviorAgent* con architetture più avanzate come Autoware o Apollo, così da valutare gli scenari generati su *full-stack ADS*. Ciò permetterebbe di misurare l’effettiva capacità dei generatori di sollecitare ADS reali, riducendo il divario con la guida reale.

Diversità semantica Oltre alla diversità calcolata in funzione di cluster, sarebbe interessante estendere l’analisi a un livello concettuale. Un LLM potrebbe essere incaricato di descrivere ciascuno scenario in linguaggio naturale, per poi trasformare tali descrizioni in embedding e valutarne la distanza semantica (es. *cosine distance*). Questo permetterebbe di cogliere differenze situazionali che, sul piano puramente metrico, resterebbero nascoste.

Metriche di realismo Infine, l'introduzione di metriche di realismo è cruciale per evitare che condizioni artificiali vengano interpretate come scenari validi. Il concetto di *Inevitable Collision State* (ICS) consente di distinguere tra situazioni realmente critiche, in cui l'ADS mantiene margini di manovra, e scenari in cui la collisione è inevitabile a priori. Integrare questa metrica significherebbe qualificare meglio i risultati, riducendo il rischio di sovrastimare la difficoltà di scenari che non avrebbero alcun corrispettivo nella guida reale.

Bibliografia

- [1] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, “Autoware on board: Enabling autonomous vehicles with embedded systems,” in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*, 2018, pp. 287–296. (Citato alle pagine iii, 7 e 11)
- [2] S. Tang, Z. Zhang, Y. Zhang, J. Zhou, Y. Guo, S. Liu, S. Guo, Y.-F. Li, L. Ma, Y. Xue, and Y. Liu, “A survey on automated driving system testing: Landscapes and trends,” *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 5, Jul. 2023. [Online]. Available: <https://doi.org/10.1145/3579642> (Citato alle pagine 5, 6, 11 e 17)
- [3] X. Zhang, J. Tao, K. Tan, M. Törngren, J. M. G. Sánchez, M. R. Ramli, X. Tao, M. Gyllenhammar, F. Wotawa, N. Mohan, M. Nica, and H. Felbinger, “Finding critical scenarios for automated driving systems: A systematic mapping study,” *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 991–1026, 2023. (Citato a pagina 6)
- [4] California Department of Motor Vehicles, “Autonomous vehicle disengagement reports,” <https://www.dmv.ca.gov/portal/vehicle-industry-services/autonomous-vehicles/disengagement-reports/>, 2021, accessed: 2025-09-12. (Citato a pagina 7)

-
- [5] G. Lou, Y. Deng, X. Zheng, M. Zhang, and T. Zhang, "Testing of autonomous driving systems: where are we and where should we go?" in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 31–43. [Online]. Available: <https://doi.org/10.1145/3540250.3549111> (Citato alle pagine 7 e 8)
- [6] H. Alghodhaifi and S. Lakshmanan, "Autonomous vehicle evaluation: A comprehensive survey on modeling and simulation approaches," *IEEE Access*, vol. 9, pp. 151 531–151 566, 2021. (Citato alle pagine 7 e 8)
- [7] A. Gambi, M. Mueller, and G. Fraser, "Asfault: Testing self-driving car software using search-based procedural content generation," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2019, pp. 27–30. (Citato a pagina 7)
- [8] S. Ulbrich, T. Menzel, A. Reschka, F. Schuldt, and M. Maurer, "Defining and substantiating the terms scene, situation, and scenario for automated driving," in *2015 IEEE 18th International Conference on Intelligent Transportation Systems*, 2015, pp. 982–988. (Citato a pagina 9)
- [9] T. Menzel, G. Bagschik, and M. Maurer, "Scenarios for development, test and validation of automated vehicles," in *2018 IEEE Intelligent Vehicles Symposium (IV)*, 2018, pp. 1821–1827. (Citato a pagina 9)
- [10] W. Ding, C. Xu, M. Arief, H. Lin, B. Li, and D. Zhao, "A survey on safety-critical driving scenario generation—a methodological perspective," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 7, pp. 6971–6988, 2023. (Citato a pagina 9)
- [11] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002. (Citato a pagina 11)

- [12] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, 2017, pp. 1–16. (Citato a pagina 11)
- [13] G. Rong, B. H. Shin, H. Tabatabaee, Q. Lu, S. Lemke, M. Možeiko, E. Boise, G. Uhm, M. Gerow, S. Mehta *et al.*, "Lgsvl simulator: A high fidelity simulator for autonomous driving," *arXiv preprint arXiv:2005.03778*, 2020. (Citato a pagina 11)
- [14] G. Kaljavesi, T. Kerbl, T. Betz, K. Mitkovskii, and F. Diermeyer, "Carla-autoware-bridge: Facilitating autonomous driving research with a unified framework for simulation and module development," 2024. (Citato a pagina 11)
- [15] M. Biagiola and S. Klikovits, "Sbft tool competition 2024 - cyber-physical systems track," in *Proceedings of the 17th ACM/IEEE International Workshop on Search-Based and Fuzz Testing*, ser. SBFT '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 33–36. [Online]. Available: <https://doi.org/10.1145/3643659.3643932> (Citato alle pagine 13, 23 e 24)
- [16] C. Xu, W. Ding, W. Lyu, Z. Liu, S. Wang, Y. He, H. Hu, D. Zhao, and B. Li, "Safebench: A benchmarking platform for safety evaluation of autonomous vehicles," 2022. [Online]. Available: <https://arxiv.org/abs/2206.09682> (Citato alle pagine 13 e 25)
- [17] J. Zhang, C. Xu, and B. Li, "Chatscene: Knowledge-enabled safety-critical scenario generation for autonomous vehicles," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 15 459–15 469. (Citato a pagina 15)
- [18] K. Elmaaroufi, D. Shanker, A. Cismaru, M. Vazquez-Chanlatte, A. Sangiovanni-Vincentelli, M. Zaharia, and S. A. Seshia, "Scenicnl: Generating probabilistic scenario programs from natural language," 2024. [Online]. Available: <https://arxiv.org/abs/2405.03709> (Citato a pagina 15)
- [19] Y. Deng, J. Yao, Z. Tu, X. Zheng, M. Zhang, and T. Zhang, "Target: Automated scenario generation from traffic rules for testing autonomous vehicles

- via validated llm-guided knowledge extraction,” 2025. [Online]. Available: <https://arxiv.org/abs/2305.06018> (Citato a pagina 15)
- [20] B.-K. Ruan, H.-T. Tsui, Y.-H. Li, and H.-H. Shuai, “Traffic scene generation from natural language description for autonomous vehicles with large language model,” 2025. [Online]. Available: <https://arxiv.org/abs/2409.09575> (Citato a pagina 15)
- [21] S. Tang, Z. Zhang, J. Zhou, L. Lei, Y. Zhou, and Y. Xue, “Legend: A top-down approach to scenario generation of autonomous driving systems assisted by large language models,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1497–1508. [Online]. Available: <https://doi.org/10.1145/3691620.3695520> (Citato a pagina 16)
- [22] N. Hanselmann, K. Renz, K. Chitta, A. Bhattacharyya, and A. Geiger, “King: Generating safety-critical driving scenarios for robust imitation via kinematics gradients,” in *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXXVIII*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 335–352. [Online]. Available: https://doi.org/10.1007/978-3-031-19839-7_20 (Citato a pagina 16)
- [23] T.-H. Nguyen, T.-G. Vuong, H.-N. Duong, S. Nguyen, H. D. Vo, T. Aoki, and T.-T. Nguyen, “Generating critical scenarios for testing automated driving systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.02574> (Citato a pagina 16)
- [24] M. von Stein, D. Shriver, and S. Elbaum, “Deepmaneuver: Adversarial test generation for trajectory manipulation of autonomous vehicles,” *IEEE Transactions on Software Engineering*, vol. 49, no. 10, pp. 4496–4509, 2023. (Citato a pagina 16)
- [25] S. Sun, Z. Gu, T. Sun, J. Sun, C. Yuan, Y. Han, D. Li, and M. H. Ang, “Drivescenenegen: Generating diverse and realistic driving scenarios from scratch,” *IEEE Robotics and Automation Letters*, vol. 9, no. 8, pp. 7007–7014, 2024. (Citato a pagina 16)

-
- [26] S. Lin, F. Chen, L. Xi, K. Xie, Y. Zheng, H. Fei, Y. Sun, and H. Zhu, "Scenariofuzz-llm: Enhancing diversity in autonomous driving scenario fuzzing with llms," in *2025 28th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2025, pp. 1581–1586. (Citato a pagina 16)
- [27] S. Lin, F. Chen, L. Xi, G. Wang, R. Xi, Y. Sun, and H. Zhu, "Tm-fuzzer: fuzzing autonomous driving systems through traffic management," *Automated Software Engg.*, vol. 31, no. 2, Jul. 2024. [Online]. Available: <https://doi.org/10.1007/s10515-024-00461-w> (Citato a pagina 16)
- [28] H. Yang, Y. Zhou, and T. Chen, "Simadfuzz: Simulation-feedback fuzz testing for autonomous driving systems," *ACM Trans. Softw. Eng. Methodol.*, Jun. 2025, just Accepted. [Online]. Available: <https://doi.org/10.1145/3744242> (Citato a pagina 16)
- [29] S. Kim, M. Liu, J. J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, "Drivefuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. ACM, Nov. 2022, p. 1753–1767. [Online]. Available: <http://dx.doi.org/10.1145/3548606.3560558> (Citato a pagina 20)
- [30] H. Wang, Z. Zhou, X. Li, S. Guo, and H. Jiang, "Learning to accelerate autonomous driving system testing," *IEEE Transactions on Reliability*, pp. 1–15, 2025. (Citato a pagina 20)